Tecnología para la Gestión II

Clase 3 — Tipos y transformación de variables (Parte I)

Francisco Villarroel Riquelme | 2026-08-19

Administración Pública · Escuela de Gobierno · Facultad de Economía y Gobierno · ADPU B012

¿Qué vimos la clase pasada?

Proyectos

Carpeta autocontenida (`.Rproj`) con rutas relativas → replicabilidad.

Carga

`install.packages()` una vez, `library()` siempre, y `read.csv()` para los datos.

Inspección

`names()`, `glimpse()`, `class()` y `view()` para conocer la base.

Hoy **usamos todo eso en la práctica** y damos el siguiente paso: entender los **tipos de variables**, corregirlos, y empezar a **transformar** los datos.

Agenda de hoy

1. Actividad

Cuenta en Posit Cloud, proyecto completo y carga de una base de datos.

2. Tipos de variables

Cuáles son, qué permite cada uno, cómo revisarlos y cómo cambiarlos.

3. Transformación

`select()`, `filter()`, `mutate()` y `case_when()`, con operadores lógicos.

Todos los ejemplos de hoy usan una **base ficticia de gestión municipal** de comunas de la Región Metropolitana.

Actividad

Manos a la obra en Posit Cloud

La actividad de hoy

Vamos a montar, desde cero y **en la nube**, un proyecto ordenado con todo lo que aprendimos: crear el proyecto, armar la estructura de carpetas, cargar una base que les voy a pasar e inspeccionarla.

Posit Cloud es la versión en el navegador de RStudio: no hay que instalar nada y el proyecto queda guardado en tu cuenta. Ideal para partir parejos.

Trabajaremos con el archivo `gestion_municipal.csv` (Está disponible en Blackboard).

Parte A — Crear cuenta y proyecto

1. Entra a **posit.cloud** y crea una cuenta gratuita (*Sign Up*). Puedes usar tu correo o tu cuenta Google.
2. En el panel, haz clic en **New Project** → **New RStudio Project**.
3. Ponle un nombre al proyecto: **clase3_gestion**.

> Verás la misma interfaz de RStudio de la clase pasada: script, environment, consola y output. La diferencia es que todo corre en el navegador.

Parte B — Carpetas, carga e inspección

4. En el panel **Files**, crea las carpetas `data`, `scripts` y `results` (*New Folder*).
5. Sube `gestion_municipal.csv` con **Upload** y déjalo dentro de `data/`.
6. Crea un script en `scripts/` y escribe:

```
install.packages("tidyverse")   # solo la primera vez
library(tidyverse)

gestion <- read.csv("data/gestion_municipal.csv")

glimpse(gestion)
```

La base con la que trabajaremos

Variable	Qué contiene
id_solicitud	Folio de la solicitud (SOL-0001…)
comuna	Comuna de la RM
fecha_ingreso	Fecha en que ingresó la solicitud
tipo_solicitante	Persona natural · Empresa · De oficio
tipo_solicitud	Alumbrado, áreas verdes, bacheo, patente…
oficina_municipal	Unidad que atiende (Obras, Tránsito, DIDECO…)
nivel_prioridad	Baja / Media / Alta
estado	1 Ingresada · 2 En proceso · 3 Resuelta · 4 Rechazada
requiere_visita	Sí / No
monto_asignado	Presupuesto asignado (CLP)
n_beneficiarios	Personas beneficiadas
dias_tramitacion	Días que tomó la tramitación
puntaje_satisfaccion	Satisfacción ciudadana (1,0 a 7,0)

Bloque 2

Tipos de variables

¿Por qué importan los tipos?

El **tipo** de una variable decide **qué puedes hacer** con ella. No puedes promediar texto ni ordenar categorías si R no sabe que lo son.

Tipo	Ejemplo	Qué puedes hacer
`logical`	TRUE / FALSE	Condiciones, filtros, contar casos
`integer`	186L	Operaciones con enteros
`double / numeric`	1.67	Sumar, promediar, comparar
`character`	"Maipú"	Contar, agrupar, etiquetar
`factor`	Baja < Media < Alta	Categorías con orden: gráficos y modelos
`Date`	2024-06-15	Ordenar por tiempo, calcular plazos

Cómo revisar el tipo

Dos funciones que ya conocemos:

```
# El tipo de una variable puntual
class(gestion$estado)     # "integer"
class(gestion$comuna)     # "character"

# El panorama completo de la base
glimpse(gestion)
```

> Recuerda las abreviaturas de `glimpse()`: `int` = integer, `dbl` = double, `chr` = character, `lgl` = logical, `fct` = factor.

Cómo cambiar el tipo

R tiene una familia de funciones `as.___()` para **convertir**:

Función	Convierte a...
`as.character()`	texto
`as.numeric()`	número (con decimales)
`as.integer()`	entero
`as.logical()`	TRUE / FALSE
`as.factor()`	factor (categoría)
`as.Date()`	fecha

Regla clave: para que el cambio quede guardado hay que **reasignar** el objeto. No basta `as.factor(gestion$comuna)`; tiene que ser `gestion$comuna <- as.factor(gestion$comuna)`.

Caso especial: el `factor` ordenado

`nivel_prioridad` es texto, y R lo ordenaría **alfabéticamente** (Alta, Baja, Media). Queremos el orden **lógico**: Baja → Media → Alta.

```
# 1) lo pasamos a factor
gestion$nivel_prioridad <- as.factor(gestion$nivel_prioridad)

# 2) le damos el orden correcto de niveles
gestion$nivel_prioridad <- factor(gestion$nivel_prioridad,
                                  levels = c("Baja", "Media", "Alta"))

table(gestion$nivel_prioridad)   # ahora aparece Baja, Media, Alta
```

> Los factores ordenados son clave para **gráficos** y **modelos**: sin ellos, "Alta" quedaría antes que "Baja" solo por el abecedario.

Diagnóstico de nuestra base

Al cargar `gestion_municipal.csv`, R clasifica varias columnas **mal**. Este es el trabajo de reclasificación:

Columna	R la lee como	Debería ser
`comuna`	character	factor
`fecha_ingreso`	character	Date
`tipo_solicitante`	character	factor
`oficina_municipal`	character	factor
`nivel_prioridad`	character	factor ordenado
`estado`	integer	factor
`requiere_visita`	character	logical
`puntaje_satisfaccion`	character	numeric

El `puntaje` viene con coma decimal ("5,3"); lo dejamos como reto opcional: `as.numeric(gsub(",", ".", gestion$puntaje_satisfaccion))`.

Bloque 3

Transformación de variables con `dplyr`

Primero: el pipe %>%

El **pipe** (`%>%`) encadena acciones sobre un mismo dataframe. Se lee como **"y luego"**.

Sin pipe

```
gestion2 <- select(gestion,
                   comuna, monto_asignado)
gestion2 <- filter(gestion2,
                   monto_asignado > 0)
```

Con pipe

```
gestion2 <- gestion %>%
  select(comuna, monto_asignado) %>%
  filter(monto_asignado > 0)
```

El pipe toma lo que está a la izquierda y lo pasa como primer argumento a la función de la derecha. Hace el código **más legible** y ordenado.

`select()` — elegir columnas

Sirve para quedarnos solo con las variables que nos interesan:

```
# quedarnos con algunas columnas
gestion %>%
  dplyr::select(comuna, tipo_solicitud, oficina_municipal, monto_asignado)

# quitar una columna con el signo menos
gestion %>%
  dplyr::select(-id_solicitud)
```

> Útil cuando la base es enorme (¡R consume mucha memoria!) o simplemente para trabajar más ordenados con las variables que necesitamos.

`filter()` — elegir filas

Selecciona **observaciones** que cumplen una condición:

```
# solicitudes ingresadas por personas naturales
gestion %>% dplyr::filter(tipo_solicitante == "Persona natural")

# solicitudes con presupuesto sobre 10 millones
gestion %>% dplyr::filter(monto_asignado > 10000000)
```

> `filter()` conserva las filas donde la condición es **verdadera** (`TRUE`). Para escribir condiciones precisas necesitamos los **operadores lógicos**.

Operadores lógicos

Operador	Significado
`==`	es igual a
`!=`	es diferente a
`>  <`	mayor que / menor que
`>=  <=`	mayor o igual / menor o igual
`&`	intersección (y)
`|`	unión (o)
`%in%`	está contenido en...

Tabla basada en tu material de RPubs y en *AnalizaR Datos Políticos* (Urdinez & Cruz).

`filter()` con condiciones combinadas

```
# prioridad alta Y que no estén rechazadas
gestion %>%
  dplyr::filter(nivel_prioridad == "Alta" & estado != 4)

# varias comunas a la vez
gestion %>%
  dplyr::filter(comuna %in% c("Maipú", "La Florida", "Puente Alto"))
```

Traducción de la primera: *"quédate con las solicitudes de prioridad Alta **y** cuyo estado sea distinto de 4 (rechazada)"*. Con `&` (y), `|` (o) y `%in%` armamos filtros tan precisos como necesitemos.

`mutate()` — crear o transformar columnas

Genera variables nuevas a partir de las que ya existen:

```
gestion <- gestion %>%
  dplyr::mutate(monto_millones = monto_asignado / 1000000,
                costo_beneficiario = monto_asignado / n_beneficiarios)
```

> `mutate()` **agrega** columnas al dataframe sin borrar las demás. Acá creamos el monto en millones y el costo por persona beneficiada.

`case_when()` — recodificar por condiciones

Ideal para pasar **códigos numéricos a etiquetas** legibles:

```
gestion <- gestion %>%
  dplyr::mutate(estado_txt = case_when(
    estado == 1 ~ "Ingresada",
    estado == 2 ~ "En proceso",
    estado == 3 ~ "Resuelta",
    estado == 4 ~ "Rechazada"))
```

También sirve para **crear un `logical`** desde "Sí"/"No":

```
gestion <- gestion %>%
  dplyr::mutate(visita_log = requiere_visita == "Sí")
```

Cada línea de `case_when()` se lee: *condición ~ valor que asigna*. Se evalúan en orden, de arriba hacia abajo.

Manos a la obra

Ejercicios con la base

Ejercicios

1. Reclasificar. Convierte `comuna`, `tipo_solicitante`, `tipo_solicitud` y `oficina_municipal` a **factor**. Convierte `nivel_prioridad` a **factor ordenado** (Baja → Media → Alta).

2. Recodificar. Crea `estado_txt` con `case_when()` (etiquetas del estado) y `visita_log` (logical) a partir de `requiere_visita`.

3. Filtrar. Quédate con las solicitudes de prioridad **Alta**, que **no** estén rechazadas y con `monto_asignado` mayor a 5.000.000.

4. Crear y seleccionar. Con `mutate()` crea `costo_beneficiario = monto_asignado / n_beneficiarios` y luego usa `select()` para quedarte solo con `comuna`, `tipo_solicitud` y esa nueva variable.

¡Buen trabajo!

La próxima clase: transformación de variables (Parte II)

Más recodificación, `ifelse()`, resúmenes por grupo y manejo de valores perdidos (`NA`).

Pauta de ejercicios solo docente

.info-box[

```r
# 1) Reclasificar
gestion$comuna <- as.factor(gestion$comuna)
gestion$tipo_solicitante <- as.factor(gestion$tipo_solicitante)
gestion$tipo_solicitud <- as.factor(gestion$tipo_solicitud)
gestion$oficina_municipal <- as.factor(gestion$oficina_municipal)
gestion$nivel_prioridad <- factor(
  gestion$nivel_prioridad,
  levels = c("Baja","Media","Alta"))

# 2) Recodificar
gestion <- gestion %>%
  dplyr::mutate(
    estado_txt = case_when(
      estado == 1 ~ "Ingresada",
      estado == 2 ~ "En proceso",
      estado == 3 ~ "Resuelta",
      estado == 4 ~ "Rechazada"),
    visita_log = requiere_visita == "Sí")
```

```r
# 3) Filtrar
gestion %>%
  dplyr::filter(nivel_prioridad == "Alta" &
                estado != 4 &
                monto_asignado > 5000000)

# 4) Crear y seleccionar
gestion %>%
  dplyr::mutate(
    costo_beneficiario =
      monto_asignado / n_beneficiarios) %>%
  dplyr::select(comuna, tipo_solicitud,
                costo_beneficiario)
```

Elimina esta diapositiva antes de proyectar si no quieres mostrar la pauta.